

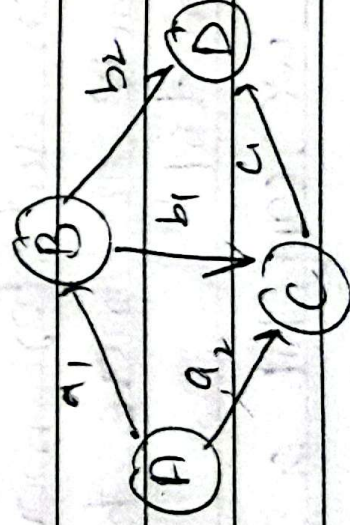
TOMORROW'S DECISION PROCESSES:-

> Uncertainty involved in this.

In Search Problem :-

state s , $\xrightarrow{\text{deterministic}}$ state $succ(s, a)$
action a

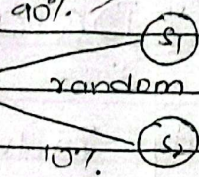
Jab bhi koi action lety we are 100% sure on which state we land.



> Whereas in this we deal with uncertainty due to real world, due to which search problem does not work for this.

Date _____

State s , action a



I might jump to s_1
or s_2 state after taking a action.

Applications:

1- Robotics:-

> actuators used for movement.

> we decide where to move but actuators can fail.

> due to this we can hit a wall.

Resource Allocation:-

> decide what to produce by looking the demands but don't know the customer demand for various products.

DECENT

Date _____

3- Agriculture:-

> decide what to plant but we don't know the weather conditions or crop yield.

VOLCANO CROSSING EXAMPLE:-

	1	2	3	4
1			-50	20
2			-50	
3	2			

• reward of 20 for scenic view.

• reward of -50

start state (1, 1)

End state (3, 1), (1, 3),

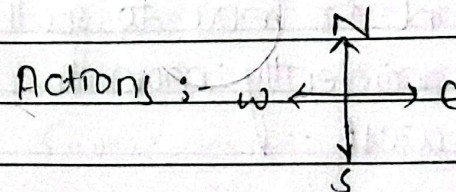
(2, 3), (1, 4)

(life ends here)

volcano.

• reward of 2

for just random view.



> Move N, move S, move E, move W.

∴ Agar uncertainty nahi then

DECENT

deterministic approach Date _____

by moving (taking certain actions) you reach to final state.

> But if uncertainty is involved:

If we have deterministic: we will get search problem sequence of actions as a solution.

In reality $\rightarrow$ we are dealing with difficult / unseen terrain.

* Slip Prob: if you want to move to south & you accidentally move to East or west.

Ex: Dice Game

For each round $r = 1, 2, 3, \dots$

- you choose stay or quit.

DECENT

Date _____

- If Quit - you get \$10 and we end the game.

- If Stay - you get \$4 and I roll a 6-sided dice.

• IF the dice result in 1 or 2 then we end the game.

• otherwise, continue to next round.

Q what is the best strategy for this game?

> Game end probability $\frac{2}{6}$ & game continue probability is $\frac{4}{6}$.

Suppose: Policy: "Stay"

DECENT

$$\therefore \frac{1}{3} \rightarrow \frac{2}{6}$$

$$\therefore \frac{4}{6} \rightarrow \frac{2}{3}$$

Date _____

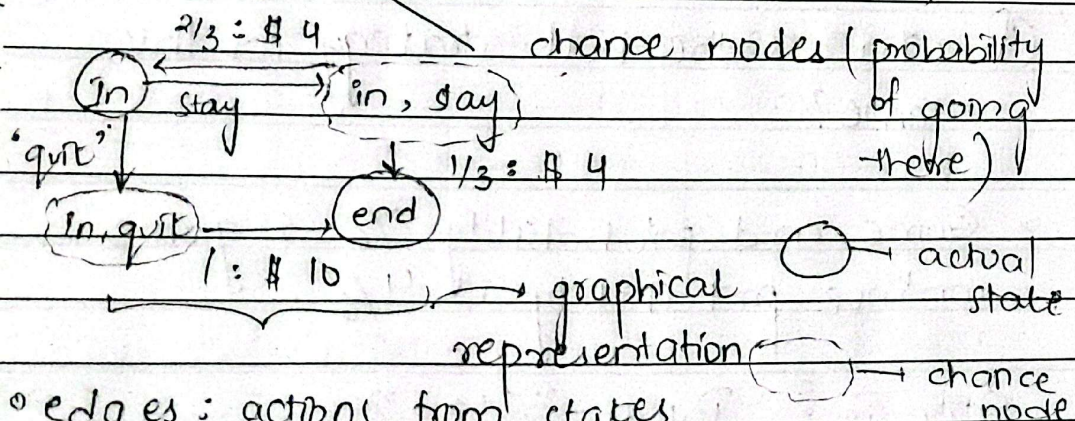
$$\frac{1}{3}(4) + \frac{2}{3} \cdot \frac{1}{3}(8) + \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{3}(12) = \$12$$

In the first round.

Suppose: Policy: "Quit"
Expected Value = 1 (10) = \$10

> MDP For Dice Game:

• nodes — states (exact state)



• edges: actions from states leading to chance nodes.

DECENT

• imaginary state: chance nodes.

Date _____

• edges coming out of chance node are possible random outcomes.

Define MDP:

- States: The set of states
- $S_{start} \in \text{States}$: starting state
- Actions: possible actions from state s
- $T(s, a; s')$ - Probability of landing on s' we take action a on state s
(if we are at state s & we take an action a what is the probability of landing on s')

• In case of search problem this was the successor func.

• $Reward(s, a, s')$ - reward of the transition (s, a, s')

DECENT

Date _____

- Is End(s) - whether at the end state.

- $0 \leq \gamma \leq 1$: discount factor

Difference in Search & MDP.

Search

MDP

(1) $\text{succ}(s, a)$

$T(s, a, s')$

↑

↑

deterministic

uncertainty

• $\text{succ}(s, a)$ is a special case of $T(s, a, s')$

$$T(s, a, s') = \begin{cases} 1 & \text{if } \text{succ}(s, a) = s' \\ 0 & \text{o/w} \end{cases}$$

In above case if above is 1 then less or more prob jany ki prob DECENT zero ho gi.

Date _____

(2) $\text{cost}(s, a)$

$\text{reward}(s, a, s')$

obj: minimize the cost.

obj: maximize the reward.

In Dice Game:

$$T(\text{in}, \text{'stay'}, \text{in}) = 2/3$$

$$T(\text{in}, \text{'stay'}, \text{end}) = 1/3$$

$$T(\text{in}, \text{'Quit'}, \text{end}) = 1$$

Trans	s	action(a)	next state(s')	$T(s, a, s')$
	in	stay	in	2/3
	in	stay	end	1/3
	in	Quit	end	1

Transition probability is the distribution over the successor states, & sum of all probabilities will be equal DECENT to 1.

$1500000 = -x$ 1500000
 $-10,00,000$ $10,00,000$
 This still good for probability Date _____ this good for search

Transition probability is distribution over successor state s' when take action a on state s .

$$\sum_{s' \in \text{state}} T(s, a, s') = 1$$

$$T(s, a, s') \geq 0$$

Transportation Example:

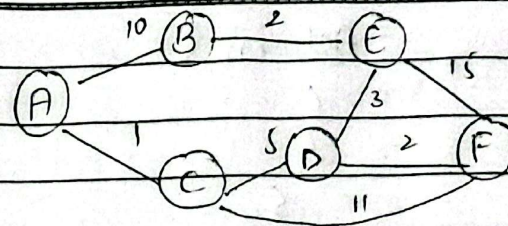
- streets with blocks 1 to n
- walking: s to $s+1$, taken $1m$
- Tram: s to $2s$ taken $2m$

How to travel from 1 to n in least amount of time?

- Tram can fail with prob = 0.5

Are obj fun ko minimize krna
 maximize krna ho to $(-)$ lg a do. DECENT

Date _____



Here A C D F

will give
cost = 8

min cost = 8

min cost = ABEF

max cost = ACDF

Now if all are
minus:

① What is a solution in MDP?

As in Search Problem → sequence of actions in a solution.

In MDP: Policy

"A policy π is a mapping from each state $s \in \text{states}$ to an action $a \in \text{Actions}$ "

Ex: In Volcano Crossing (my policy can be DECENT

Date _____

State	Action
(1,0)	S
(2,1)	E
(1,2)	S
(1,3)	N

$\pi : S \rightarrow A(S)$

har aik state pay hum ny
kon sa action lena hali

LECTURE # (MUS)

LECTURE # 24th Dec, 24

- Algorithm jo achi policy nikal kiy dy.
- Utility: sum of discounted reward
jo aik dfa wn kmy say
milti. (policy ko
wn kr ky)

DECENT

Date _____

Expected Utility: if you run it multiple times.

So far:

→ Given policy π : we can then use
policy evaluation
algorithm to evaluate the
policy.

Now:

How to find policy π ?
(we want max reward)

- Find a policy to get max expected utility. $E(U)$
- : How to get optimal value & policy.

- Try to get directly max $E(U)$.
optimal value here means: max value
obtained by any policy.

DECENT

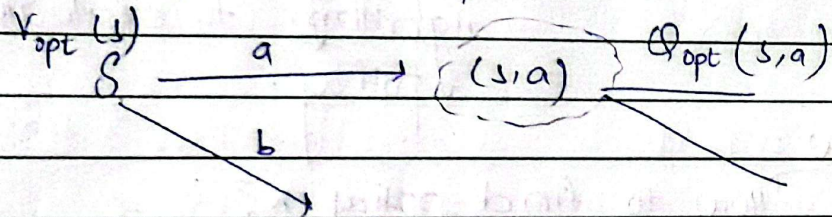
main samjhi nahi a rha!

Date _____

Optimal value & Optimal Q value:-

Prev - $V_{\pi}(s), Q_{\pi}(s,a)$

Now $V_{opt}(s), Q_{opt}(s,a)$



In order to find V_{opt} we use Q_{opt}
How to find $V_{opt}(s) = ?$

Optimal value; from state s ..

$$V_{opt} = \begin{cases} 0 & \text{if End} = T \\ \max_{a \in \text{Action}(s)} Q_{opt}(s,a) & \text{o/w} \end{cases}$$

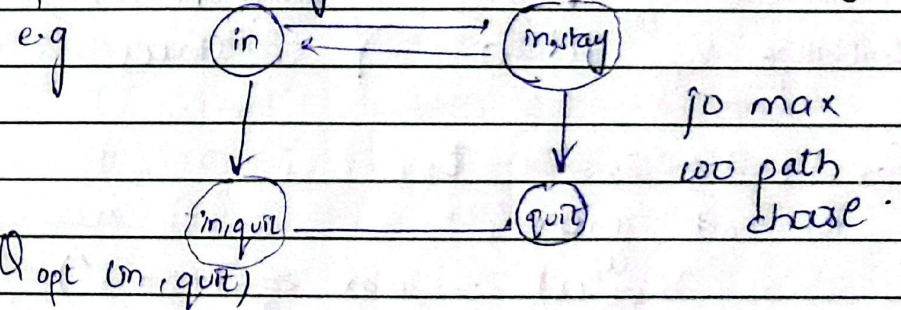
DECENT

chance node is key bad probabilistic

Date _____

$$Q_{opt}^{\pi}(s,a) = \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma V_{opt}^{(u-1)}(s')]$$

Given Q_{opt} we can get the optimal policy?



e.g volcano crossing:

	1	2	3	4	
1	0	1	1/5, 1	1/5, 1	choose max value
2	4	9	1/5, 1	5	
3	1/5, 1	2	4	1	

DECENT

Mathematically: $\pi_{opt}(s) = \arg \max_a V_{opt}(s, a)$ which max

value
ways
found.

used to find optimal policy.
Algorithm: Value Iteration.

(very old algo) - (proposed by
Bellman 1957)
Initialize $V_{opt}^{(0)}(s) = 0 \quad \forall s \in \text{states}$

for $t = 1, 2, \dots, T$
for each state s :

$$V_{opt}^{(t)}(s) = \max_a \sum_{s'} T(s, a, s') V_{opt}^{(t-1)}(s')$$

$a \in \text{Actions}(s)$

$$[R(s, a, s') + \gamma V_{opt}^{(t-1)}(s')]$$

$$(\gamma V_{opt}^{(t-1)}(s'))$$

Time complexity:

$$O(|S| \cdot |A| \cdot |S'|)$$

taking max over action we can find π_{opt}

DECENT

Episode:

Policy π : stay
[in: stay, 4, in: stay, 4, end]

These paths are
called episodes

- start state, Action (s)

- $T(s, a, s')$ both are unknown
- $R(s, a, s')$ is real world.

isend (s)

Reinforcement Learning:

(offline)

MDP

RL

- Have mental model of how the world works.

works.

- find the policy that performs actions in the real world

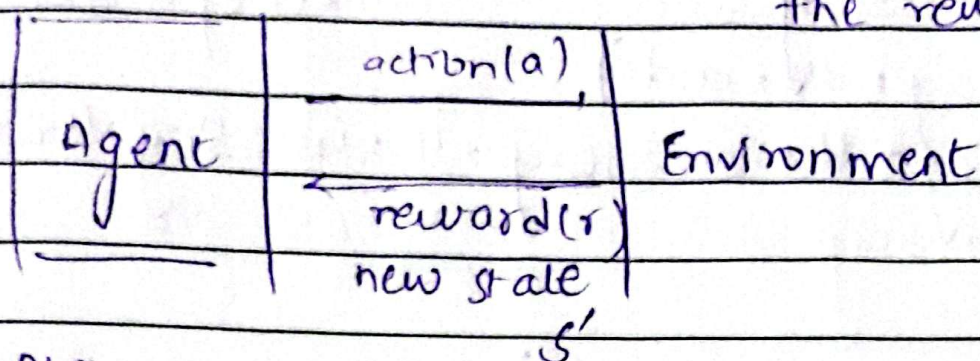
DECENT

reward.

Date

Date _____

RL Framework: to find out & collect the reward.



Algo :

for $t = 1, 2, 3, \dots$

- Choose action $a_t = \pi_{act}(s_{t-1})$ [How?]
- Receive Reward r_t and observe a new state s_t .
- Update parameters (transition & reward updation) [How?]

MONTÉ - CARLO METHODS :-

→ Model based Monte Carlo :

↳ Estimates the MDP $T(s, a, s')$ and $R(s, a, s')$ (based on data.)

DECENT

Data:

s_0, a_1, r_1, s_1
 a_2, r_2, s_2
 a_3, r_3, s_3

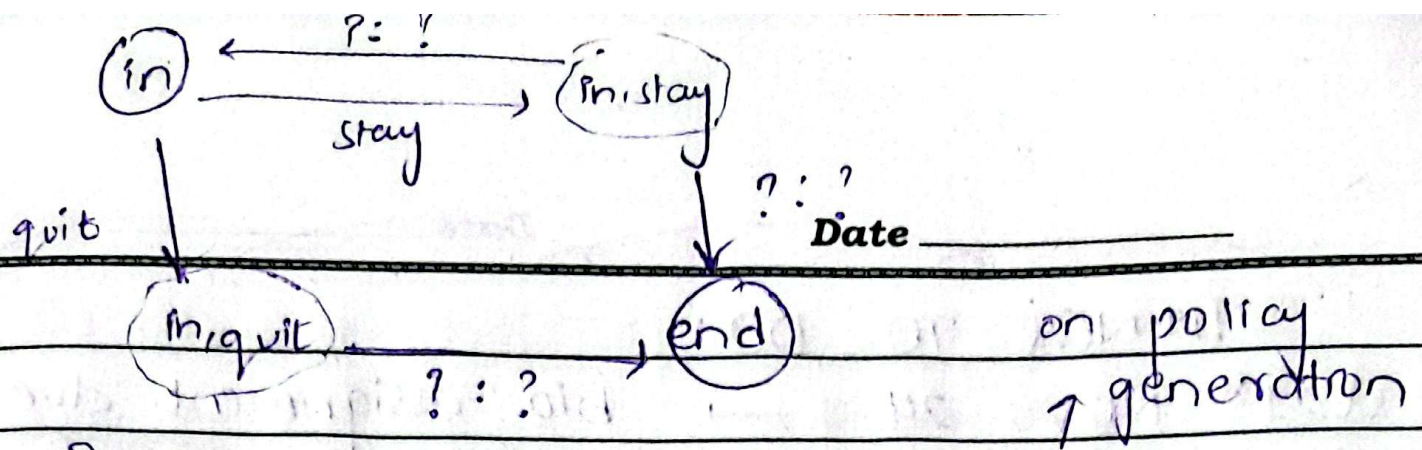
} reward

Transition : $\vec{T}(s, a, s') = \frac{\# \text{ Times } (s, a, s')}{\# \text{ Times } (s, a)}$

Times (s,a) occurs

Reward $(s, a, s') = \checkmark$ in (s, a, s')
 actual

Example : Dice Game.



Policy : stay (Data generation based on Data :- 1 :)

a policy is termed as

[in ; stay , 4 , in ; stay , 4 , in ; stay , 4 , as]

[in ; stay , 4 , end]

$T(\text{in}, \text{stay}, \text{in}) = \frac{3}{4}$ } Transition probability based on first episode.

$R(\text{in}, \text{stay}, \text{in}) = 4 \$$

$R(\text{in}, \text{stay}, \text{end}) = 4 \$$

Data :- 2

[in ; stay , 4 , in ; stay , 4 , end]

$T(\text{in}, \text{stay}, \text{in}) = \frac{4}{6}$

$T(\text{in}, \text{stay}, \text{end}) = \frac{2}{6}$

//
//

- Transition probability & reward

we don't know in real world

estimate $\leftarrow$ so we

T.P & reward. we use Monte Carlo Methods

Exploration Problem

(every state should be explored)

- we need optimum policy which can be derived from queue values $\rightarrow$ Model free learning.

Model Free Monte Carlo:-

$$\hat{Q}_{opt}(s, a) = \sum_{s'} \frac{1}{T(s, a, s')} \left[\text{Reward}(s, a, s') + \gamma \hat{V}_{opt}(s') \right]$$

meaning

- I will focus on directly $\hat{Q}_{opt}$ rather than $\hat{T}$ or Reward.
- Try to estimate $\hat{Q}_{opt}$ directly.

$\rightarrow$ Following a policy $\pi \rightarrow$ given Data.

$S_0; a_1, r_1, S_1; a_2, r_2, S_2; a_3, r_3, S_3; \dots$

✓ Utility: accumulated reward by a certain random variable policy.

we can find utility at time t

$$U_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$
 Discounted sum of reward.

Estimate $\hat{Q}_{\pi}(s, a)$ = average of U
 where $s_{t-1} = s$ → previous time
 $a_t = a$ → pe ki state main hai.

Policy $\pi(s)$ → want to execute this.

----- $s, a, \dots$ ----- $= U_t =$

----- $s, a, \dots$ ----- $= U_t =$

----- $s, a, \dots$ ----- $= U_t =$

$\hat{Q}_{\pi}(s, a) = ? \text{ avg } U_t$

e.g Policy π : stay $\gamma = 1$

• [in ; stay, 4, end] $= U_t = 4$

• [in ; stay, 4, in ; stay, 4, in ; stay, 4, end] $\xrightarrow{U_t=12}$

• [in ; stay, 4, in ; stay, 4, end] $\rightarrow U_t = 8$

• $Q_{\pi}(\text{in, stay}) = \frac{4+12+8}{3} =$

on-policy & off-policy algorithm

↓
 khas policy
 hoti hai base
 pe data generate
 hota hai which
 is then used.

↓
 ko khas policy
 ko follow nhi kr
 rhy.
 e.g

M.B Monte Carlo

e.g
 M.B Monte Carlo

• M.F is on
 policy bcz based
 on policy data
 generate hota.

Model Free M.C :

$\hat{Q}_{\pi}(s, a) = \text{avg } U_t$

$\hat{Q}_{\pi}(s, a) = (1 - \eta) \hat{Q}_{\pi}(s, a) + \eta U$

where $\eta = \frac{1}{1 + (\# \text{ of updates to } (s, a))}$

e.g
 Data: $U = 4, 1, 1, \dots (s, a)$ ignoring (s, a)

$$\hat{Q}_\pi(s,a) = (1-\eta)\hat{Q}_\pi(s,a) + \eta v$$

$$= (1-1)0 + 1(0) = 0$$

$$\eta = 1$$

$$\hat{Q}_\pi(s,a) = \frac{1}{2}(0) + \frac{1}{2}(1)$$

$$= \frac{1}{2}(0) + \frac{1}{2}(1)$$

$$= \frac{1}{2}(0) + \frac{1}{2}(1) \quad \eta = \frac{1}{2}$$

$$= \frac{1}{2}$$

$$\hat{Q}_\pi(s,a) = (1-\frac{1}{3})\frac{1}{2} + \frac{1}{3}(1)$$

$$= \frac{2}{3} + \frac{1}{3}$$

$$= \frac{3}{3} \quad \eta = \frac{1}{3}$$

Equivalence Notation (G.D) :-

$$\hat{Q}_\pi(s,a) = \hat{Q}_\pi(s,a) - \eta \left[\hat{Q}_\pi(s,a) - v \right]$$

prediction $\uparrow$ target $\nearrow$

learning rate/step size $\downarrow$

1	X	X	-50	50
2	X	X	-50	X
	X	X	X	

$$Q_\pi((1,1), N) = \underline{\hspace{2cm}}$$

$$Q_\pi((1,1), S) = \underline{\hspace{2cm}}$$

$$Q_\pi((1,1), E) = \underline{\hspace{2cm}}$$

$$Q_\pi((1,1), W) = \underline{\hspace{2cm}}$$

jahan max us direction main jain gay

Q-Learning algorithm:

on each (s, a, r, s')

"state s say action a lety reward r milta or s' state pay chly jaty"

$$\hat{Q}_{opt}(s,a) = (1-\eta)\hat{Q}_{opt}(s,a) + \eta(r + \hat{V}_{opt}(s'))$$

$$\hat{V}_{opt}(s') = \max_{a' \in \text{Action}(s')} \hat{Q}_{opt}(s', a')$$

Initially the values will be zero in prblm which will be updated sath sath

a' = action which gives the max value

X	X		
X	X		
X	X		

REINFORCEMENT LEARNING:-

- covering the unknowns for $t = 1, 2, 3, \dots$

Choose action $a_t = \pi_{act}(s_{t-1}) \rightarrow$ How?

(real world main kon sa action lam gay).

Rec reward r_t & observe state & update parameters (how?).

Which policy π_{act} to use?

Attempt 1:

$$\text{set } \pi_{act}(s) = \arg \max_{a \in \text{Actions}(s)} \hat{Q}_{opt}(s, a)$$

(Not doing exploration only exploitation)
greedy approach.

- best solution nah milly, local mil jaye, might get stuck in local minima.

Attempt 2:

No exploitation, only exploration

$\pi_{act}(s) = \text{random}$ (Not considering from Actions(s) $\hat{Q}_{opt}$ can get in exploration)

- Not thinking of converging & going in direction of best sol.

Solution:-

* Trade off b/w Exploration & Exploitation.

Epsilon Greedy Policy Used to achieve this)

$$\pi_{act}(s) = \begin{cases} \arg \max_{a \in \text{Actions}(s)} \hat{Q}_{opt}(s, a) & \text{prob} = 1 - \epsilon \\ \text{Random from Actions}(s) & \text{prob} = \epsilon \end{cases}$$

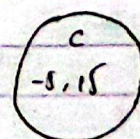
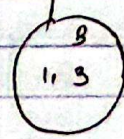
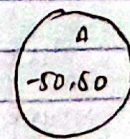
- balance = $\epsilon = 0.5$
- if 0 then exploitation otherwise exploration.

LECTURE #

7th Jan, 25

ADVERSIAL GAMES:- a game having opponent-computer

A Simple Game.



- you choose one of the bin
- I choose no from that bin.
- Your goal is to maximize the chosen no.

↳ Choose bin B bec os main sany positive hain. opponent ziyada sy ziyada ldy ga -ive nhi.

B → worst enemy

A → If game fixing

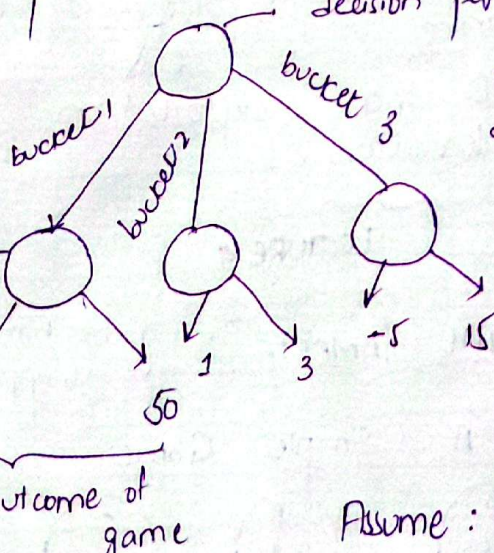
C → If random behaviour.

• Similar to state based models.

• Game Tree:

- Each node is a decision point.
- Each root to the leaf node path is a possible outcome of the game.

decision point -



- Game → Players
- State of the game is fully observed (you can see the full view of that game)

Assume: Two Player Game:

• Zero sum Game: what

you gain & what opponent gain when added gives/ equals to zero. (Agent, Opponent)

Zero sum Games -

↳ "Utility of the Agent is -ive of the utility of that of opponent"

• TWO PLAYER ZERO SUM GAME -

Formal Definition:

S_{start} - Start State

Action(s) - possible Actions from state s.

Succ(s, a) - known (what is next state)

IsEnd(s)

Utility(s) - Agent's utility for endstate(s).

Player(s) ∈ Players (who has the turn or player who control the state s)

Diff b/w - State based Models & Adversarial Games:

- In games each state has designated player (which specifies whose turn it is).
- Instead of cost on edges, we have utility at the end state. (rewards & cost).

• In this case utility is already given. (for game).

Example:-

Chess Game:

Players $\in \{ \text{white, black} \}$

State s : Position of all pieces, whose turn it is

Actions (s) : legal chess moves at state s

IsEnd(s) : whether is checkmate / draw

Utility(u) $\rightarrow +\infty$ if white wins
 $-\infty$ if black wins.

Policies :-

- state to action mapping.
- all possible state by action define to strategy (chain.)

In case of Chess Game:

• Deterministic Policy $\pi_p(u) \in \text{Action}(s)$

• Stochastic Policy $\pi_p(s, a) \in [0, 1]$
 $\rightarrow$ action on state (s) returns probability.
 (probability of a player p taking an action on state (s))

• Kon sa action state s py perform krna hai. (Actions that player p takes in state s).

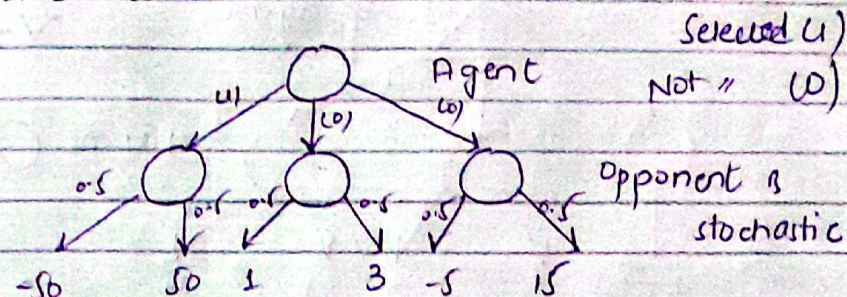
Game Evaluation Example:-

Policy:

$\pi_{\text{agent}}(s) \in A$ (will select bin A on state s)

$\pi_{\text{opp}}(u) = 1/2$ for $a \in \text{Actions}$

Game Tree:

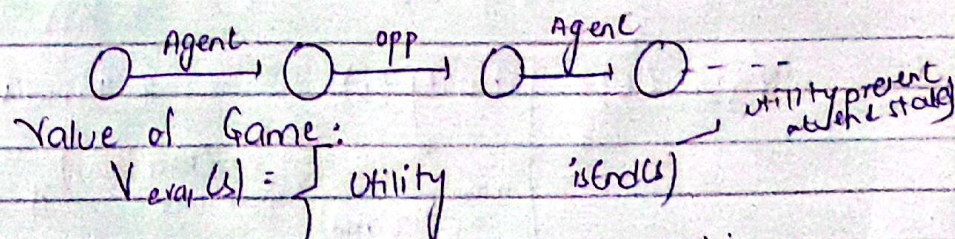


$$\begin{aligned} V_{\text{eval}}(\text{Start}) &= 1 [0.5(-50) + 0.5(50)] \\ &= +0 [0.5(1) + 0.5(3)] \\ &= +0 [0.5(-5) + 0.5(15)] \end{aligned}$$

(bottom-up approach)

$$V_{\text{eval}}(\text{Start}) = 0$$

Game Evaluation Recurrence:



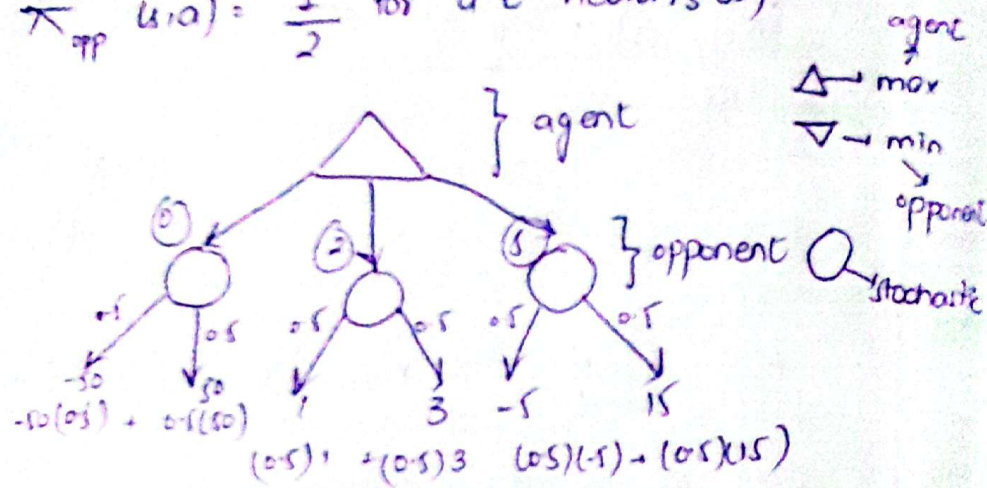
$$\text{Player}(s) = \text{Agent} \quad \left\{ \sum_{a \in \text{Actions}(s)} \pi_{\text{agent}}(s, a) V_{\text{eval}}(\text{succ}(s, a)) \right.$$

$$\text{Player}(s) = \text{opponent} \quad \left\{ \sum_{a \in \text{Actions}(s)} \pi_{\text{opp}}(s, a) V_{\text{eval}}(\text{succ}(s, a)) \right.$$

Expectimax Example:-

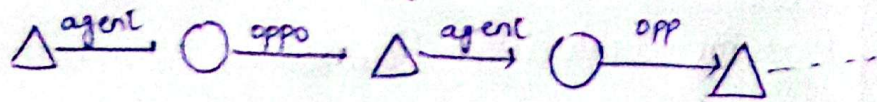
- Hum maximize and chah rhy. opponent ko stochastic (random) behaviour hai.

$$\pi_{\text{pp}}(a|s) = \frac{1}{2} \text{ for } a \in \text{Actions}(s)$$



So we should choose bucket C bcz from that direction you will get maximum value.

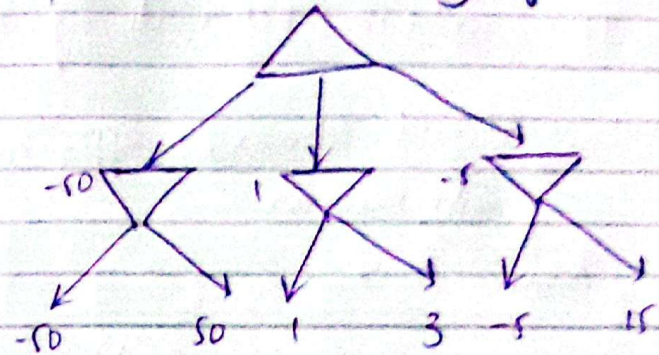
Expectimax Recurrence:-



$$V_{\text{eval}}(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ Ends} = T \\ \max_{a \in \text{Actions}(s)} V_{\text{expectmax}}(U(s,a)) & \text{if Player}(s) = \text{agent} \\ \sum_{a \in \text{Actions}(s)} \pi(s,a) V_{\text{expectmax}}(U(s,a)) & \text{if Player}(s) = \text{oppo} \end{cases}$$

MiniMax Example:-

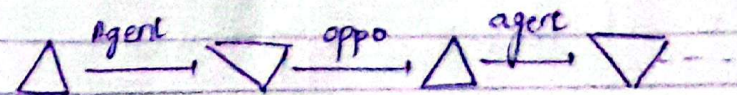
agent $\rightarrow$ maximizing
opponent $\rightarrow$ minimizing



$$V_{\text{minimax}}(\text{Root}) = 1$$

Recurrence:-

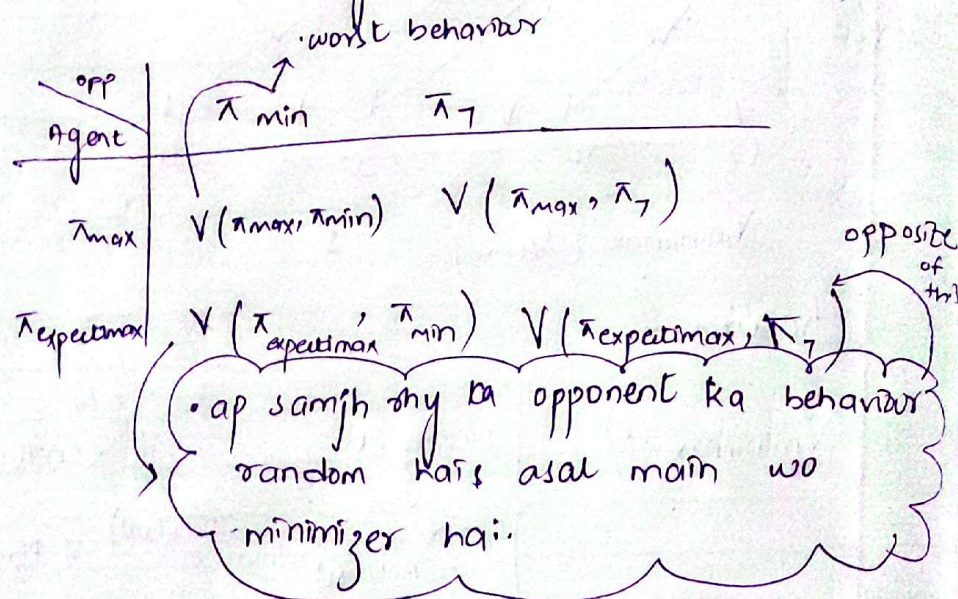
$$V_{\text{minimax}}(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ Ends} \\ \max_{a \in \text{Actions}(s)} V_{\text{minimax}}(U(s,a)) & \text{if Player}(s) = \text{agent} \\ \min_{a \in \text{Actions}(s)} V_{\text{minimax}}(U(s,a)) & \text{if Player}(s) = \text{oppo} \end{cases}$$



Properties of Minimax:-

- $V_{\text{expectimax}}$ - Agent is following max e.g. π_{max}
opp - random behaviours e.g. π_7

- V_{minimax} - Agent (π_{max}), opp- (π_{min})



- Minimax Property - 1: $\nearrow$ strategy best if

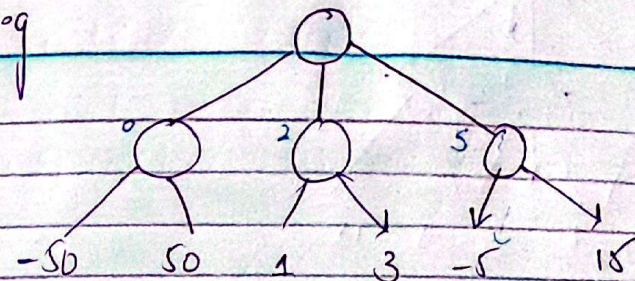
$\rightarrow$ Best against minimax opponent.

- if ap ki maximizer - opponent minimizer

using minimax then

$$V(\pi_{\text{max}}, \pi_{\text{min}}) \geq V(\pi_{\text{agent}}, \pi_{\text{min}}) \text{ for all agents}$$

e.g.



$$V(\pi_{\text{max}}^{\text{hum}}, \pi_{\text{min}}^{\text{opponent}}) = 1$$

$$V(\pi_{\text{expectimax}}, \pi_{\text{min}}) = -5$$

I am expecting opponent random behave kya ga (minimizer nhi hai), main maximizer

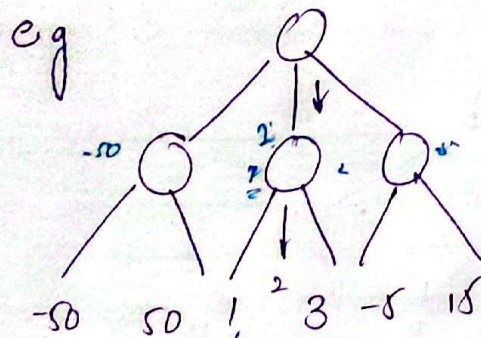
(Is kya hum node 3 pe ja shy but wo minimizer hai so wo humain -5 de ga instead of 15)

Minimax Property - 2:-

$\rightarrow$ lower bound against any opponent.

$$V(\pi_{\text{max}}, \pi_{\text{min}}) \leq V(\pi_{\text{max}}, \pi_{\text{opp}}) \text{ for all } \pi_{\text{opp}}$$

(either minimizer or give better results for you)



$$V(\pi_{\max}, \pi_{\min}) = 1$$

$$V(\pi_{\max}, \pi_7) = 2$$

lets suppose opp
is changing the
policy from
 $\pi_{\min}$ to π_7 .

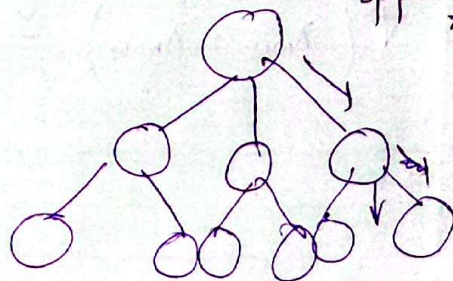
(Random main hamesha
mean ata, hum opp ko
wast soch ka mid main
gaty or wo minimizer ki jagha
random behave krta)

• Minimax Property - 3 :-

→ Not optimal if opponent is known.

$$V(\pi_{\max}, \pi_7) \leq (V(\pi_{\text{expectimax}}, \pi_7)) \text{ for opponent } \pi_7.$$

$$V(\pi_{\max}, \pi_7) = 2$$

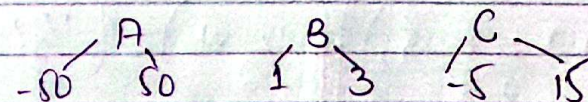


hamain pata hum maximizer &
opponent random behave kr rha.

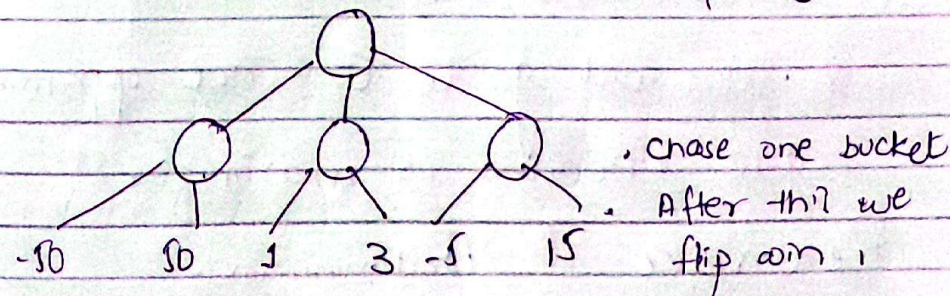
$$V(\pi_{\text{expectimax}}, \pi_7) = 5$$

MODIFIED GAME:-

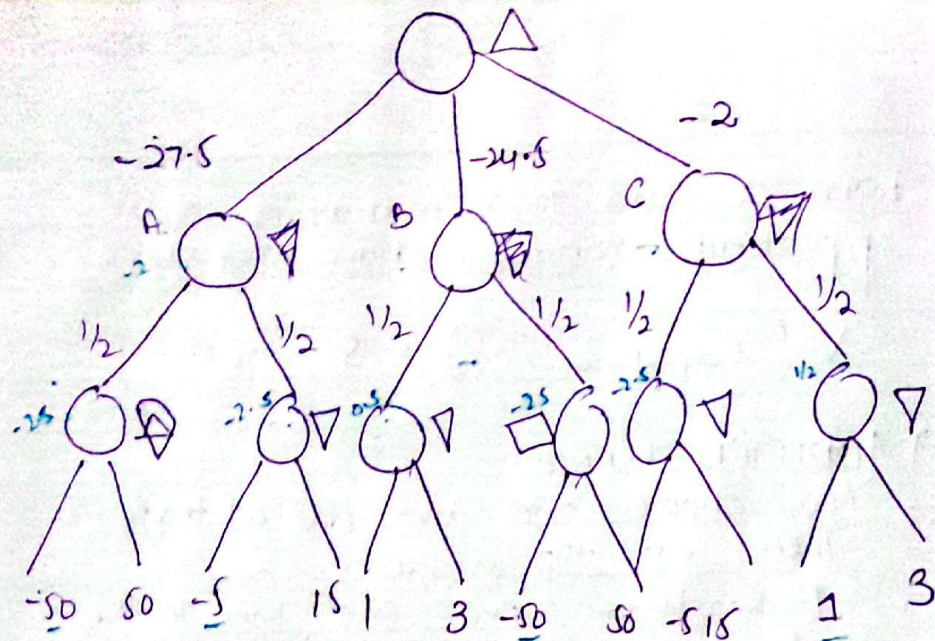
- You choose one bin out of three.
- Then we flip a coin.
- If heads — move one bin to the left (with wrap around)
- I choose a number from that bin.
- Our goal is to max the chosen no.



e.g. :- expectiminimax
 $\pi_{\min}(a) = 1/2$ for $a \in \{0, 1\}$ either head or tail.



Hum maximizer, opp min, nature random



$\checkmark$ (expectiminimax) $(U_{Start}) = ?$
 (nature) random $\uparrow$ opp $\uparrow$ us

$$\left[\left(\frac{1}{2} * -50 + \frac{1}{2} * -5 \right) \right] \left[\left(\frac{1}{2} * 1 \right) + \left(\frac{1}{2} * -50 \right) \right]$$

$$\left[\left(\frac{1}{2} * -5 \right) + \left(\frac{1}{2} * 1 \right) \right]$$

So we will go to C since opp-minimizer
nature random, us maximizer.

Recurrence of Expectiminimax:-
 $\Delta \rightarrow O \rightarrow \nabla$

$$V_{\text{expectimax}}(u) = \begin{cases} \text{Utility } (u) & \text{isEnd } (u) \\ \max_{a \in \text{Action } (u)} V_{\text{expectiminimax}}(u_{\text{succ}}(u, a)) & \text{Player } (u) = \text{Agent} \\ \min_{a \in \text{Action } (u)} V_{\text{expectiminimax}}(u_{\text{succ}}(u, a)) & \text{Player } (u) = \text{opp} \\ \sum_{a \in \text{Action } (u)} \pi_{\text{opp}}(u, a) V_{\text{expectiminimax}}(u_{\text{succ}}(u, a)) & \text{Player } (u) = \text{coin} \end{cases}$$

How to calculate value of Game?
Exhaustive Search?

Computation:-

Ex: chess game
branching $\rightarrow b$, depth $= d$ (2d)

Space $O(b^d)$
 Time $O(b^{2d})$

are barf
 man
 are
 barf
 dony
 player
 K

Chess Game $b \approx 35$, $d \approx 50$

har level

pay itri branchy

Go Games:-

$$b = 361, d \approx 100$$

Very large computation but we want to speed up we can't use minimax directly! Two strategies used to overcome this are:

→ Evaluation Function:- Use domain knowledge, compute approximate answer.
e.g. season till 3-ply

→ Alpha Beta Pruning:- General purpose, compute exact answer.

• Depth limited tree used to Evaluation function.

• what should be value for limit

↳ and according to that you choose limit take jana hai

$$V_{\text{minimax}}(s, d) = \begin{cases} \text{Utility} & \text{if } \text{end}(s) \\ \text{Eval}(s) & d = 0 \end{cases}$$

$d = \text{limit}$

$$V_{\text{minimax}}(s, a, d) = \begin{cases} \max_{a \in \text{Action}(s)} & \text{Player = Agent} \\ \min_{a \in \text{Action}(s)} & \text{Player = opp.} \end{cases}$$

$\text{Eval}(s) \rightarrow$ estimate of $V_{\text{minimax}}(s)$

↓
like future cost in DP & A*

• $\text{Eval}(s)$ in Chess Game:-

$\text{Eval}(s) = \text{material} + \text{mobility} + \text{king safety} + \text{central control}$

$\begin{matrix} \uparrow & \uparrow \\ \text{my} & \text{opp} \end{matrix}$

$$\text{material} = 100^{100} \left(\begin{matrix} \uparrow \text{my} \\ B - K' \end{matrix} \right) + \text{either my or opp}$$

$\begin{matrix} \uparrow & \uparrow \\ \text{my} & \text{opp} \end{matrix}$

$$9 (R - R') + 5 (B - B') + \text{king is taken}$$

$$3 (B - B' + N - N') + \text{checkmate, game end}$$

$$1 (P - P')$$

$\begin{matrix} \uparrow & \uparrow \\ \text{my} & \text{opp} \end{matrix}$

$$\text{mobility} = 0.1 (\text{num of legal moves} - \text{no. of legal moves'})$$

$\begin{matrix} \uparrow & \uparrow \\ \text{my} & \text{opp} \end{matrix}$

Alpha Beta Pruning:

• Exact results given.

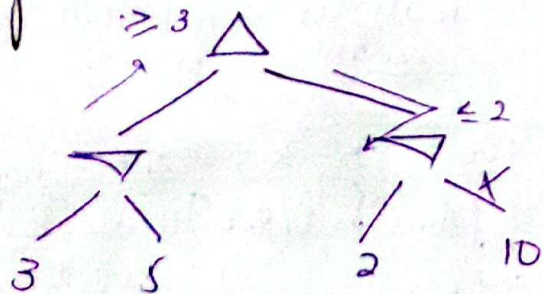
Assume: Choose A or B with max value:

A = [3, 5] B = [5, 100]

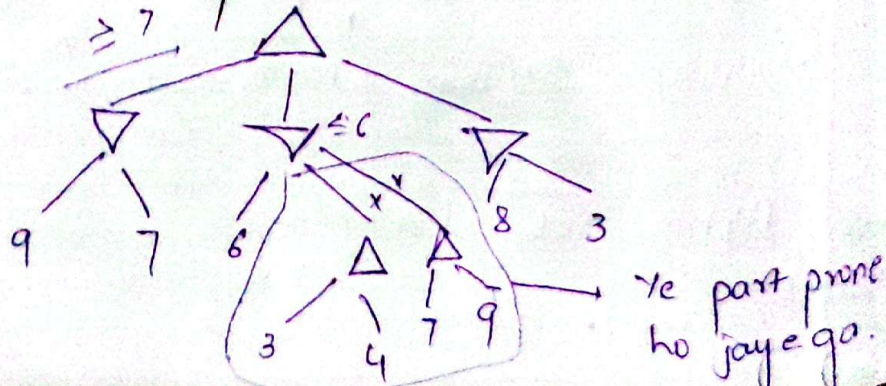
- Since A ka max B ka min same so B is better even for minimum case. so you don't look at A.
- wo wali trees ki branches explore hi nah krain jin ka faida nhi.

3 ~ 5 ~ 100
Non-overlapping ranges

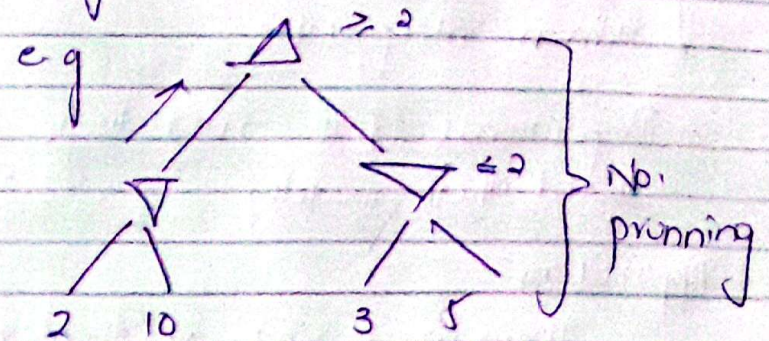
Pruning of Game Trees:



Another Example:-



Ordering of the nodes may disturb the pruning / ordering matters.



Complexity:

worst ordering $O(2^{20})$ time

Best ordering $O(2^{10 \times 0.5})$ time

Random ordering $O(2^{20 \times 0.75})$ time

LECTURE #

16th Jan, 25

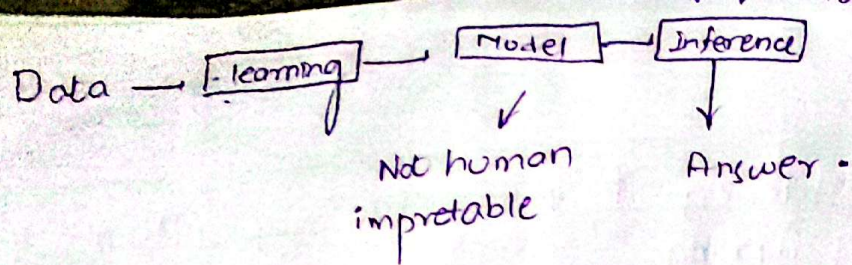
LOGIC BASED MODELS:-

↳ knowledge representation in this.

Question:- if $x_1 + x_2 = 10$ and $x_1 - x_2 = 4$ then what is x_1 ?

Two Mechanisms possible:

↳ By Search ↳ By solving these eq.



• Logic Based Models:-
Human interpretable.

- Propositional / first order logic used to build these models.

Applications:

- Theorem proving, verification reasoning.
- think in terms of logical formulas and inference rules.

Problems:

1- In sy 10 rules nully no deterministic noty (don't handle uncertainty)
Sol (Probability) -

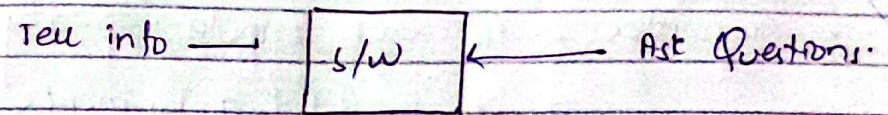
2- Rule based: did not allow fine tuning using data.
knowledge to data we formulate rules based.
Sol (Machine Learning)

Strengths:

1- It provides expressiveness in a compact way.

Motivational Example:

→ Personal Assistant Software (for university)



- we need to digest heterogeneous information.
- Question should be answered with reasoning deeply.

Q Why not natural language?

→ us main ambiguity. not hai / informal

e.g. A dime is better than a nickel.
A nickel is better than a penny.

• Therefore dime is better than penny.

- ✗ • Penny is better than nothing.
- Nothing is better than world peace.

Therefore penny is better than world peace.

Logic Based Model:

Goals:-

- Represent knowledge about the world.
- Reason with that knowledge.

Ingredients of logic:-

- syntax: set of valid formulas.
e.g. Rain $\wedge$ wet.
- semantics: for each formula, specify set of models (assignments configuration of the world)

(Rain) (wet)

	0	1
0	0	0
1	0	///

Inference Rules:-

- Ex Rain $\wedge$ wet
↳ derive Rain, wet
can't derive not rain.

Syntax $\nleftrightarrow$ Semantics:

$$\underline{2+3}$$

$$\underline{3+2}$$

Two diff syntax but same semantics.

Same syntax but diff semantics.

e.g. Python $(2.7) = 3/2 \rightarrow 1$ (int div)

Python $(3) = 3/2 \rightarrow 1.5$ (float div)

Propositional logic:-

Syntax:-

- symbols (A, B, C, ...) — atomic formulas.
- logical connections: $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$

Build formulas Recursively. f, g

- $\neg f$
- $f \wedge g$
- $f \vee g$
- $f \rightarrow g$
- $f \leftrightarrow g$

- formulas are just syntax they don't convey any meaning.

Models:-

model in propositional logic is an assignment of truth values to propositional symbols.

e.g.

3 - prop symbols (A, B, C)
= 2^3 models

$A = 0, B = 0, C = 0$

$A \wedge B \rightarrow C$

Interpretation Function :-

let f be a formula.

let w be a model

$I(f, w)$ return

: True (1) [if w satisfies f]

: false (0) [if "donot"]

e.g $f = (\neg A \wedge B) \leftrightarrow C$

Model - $w = \{A=1, B=1, C=0\}$

Interpretation -

$I((\neg A \wedge B) \leftrightarrow C, w) = 1$

$I((\neg A \wedge B), w) = 0$ $I(C, w) = 0$

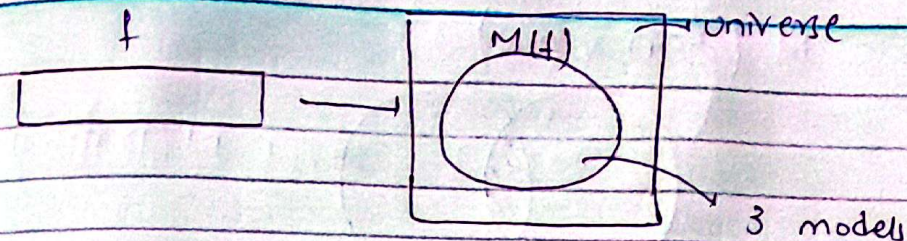
$I(\neg A, w) = 0$ $I(B, w) = 1$

$I(A, w) = 1$

• Ho sikta hai multiple models us ko satisfy kr rhy hon.

e.g $f = A, B, C$
 $w = 1, 2, 1$

model
hamam values
bata tha kaha
hai



* Formula represent set of models ko satisfy kr rhy hon.

e.g

formula (f) = Rain $\vee$ wet

Models:

wet

		0	1
Rain	0		///
	1	///	///

3 models can be represented by this formula

Knowledge Base (KB)

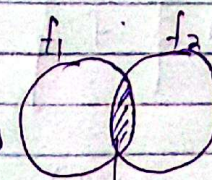
KB is set of formulas representing their conjunction.

$M(KB)$

KB ko kon kon sy models satisfy kr rhy hon.

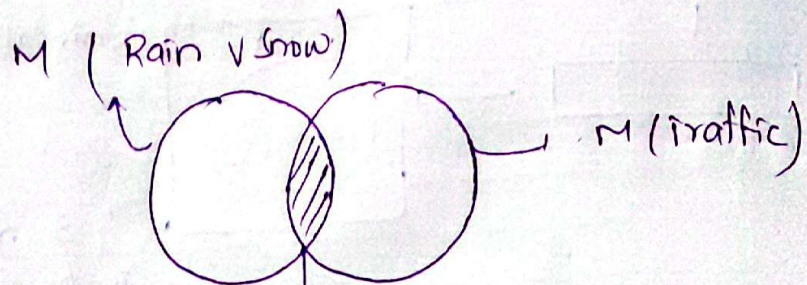
$M(f)$

is formula ko kon sy models satisfy kr rhy



KB represents this

eg KB = { Rain $\vee$ Snow, Traffic }



$M(KB)$

	0	1
0	0	1
1	1	1

$M(Rain)$

0	1	1
1	1	1

$M(Rain \rightarrow wet)$

	0	1
0	1	1
1	1	1

$M(Rain, Rain \rightarrow wet)$

	0	1
0	0	1
1	1	1

Adding to the KB :-

Adding :- (formula)

$$KB \longrightarrow KB \cup \{ \cdot f \}$$

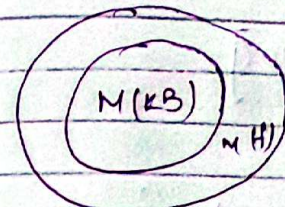
Shrink Model:

$$M(KB) \longrightarrow M(KB) \cap M(f)$$

- formula
add model
shrink.

Entailment:-

KB entails f ($KB \models f$)
iff $M(KB) \subseteq M(f)$



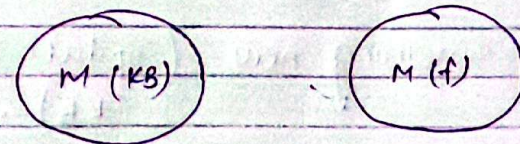
eg Rain $\wedge$ Snow $\models$ Snow

	Snow	
	0	1
Rain	0	1
	1	1

$\supseteq$

	Snow	
	0	1
Rain	0	1
	1	1

Contradiction:-



KB contradicts f iff

$$M(KB) \cap M(f) = \emptyset$$

Rain $\wedge$ Snow contradicts $\sim$ Snow

	Snow	
	0	1
Rain	0	1
	1	1

$\cap$

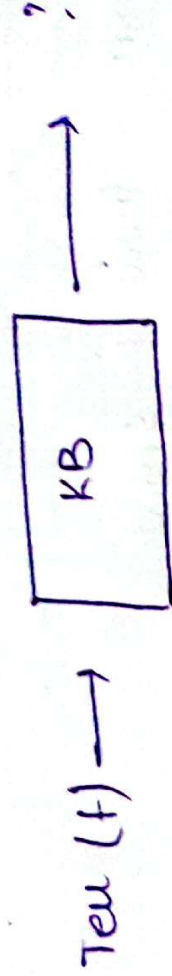
	Snow	
	0	1
Rain	0	1
	1	1

Contingency :-



$$\phi \subseteq M(KB) \cap M(H) \subseteq M(KB)$$

Teu Operation :- (learning)



Possible outcomes: (ex: It is raining)

- Already know that (Entailment)
- Don't Believe that (Contradiction)
- learned something new (update the $KB \leftrightarrow \text{contingency}$)

Ask Operation :- (ex: Is it raining)
↳ inference

- Yes
- No
- I don't know